

Gobuster

Content & Virtual Host Discovery — Web Application Penetration Testing Notes

Eyad Islam El-Taher

Contents

1	Introduction	2
2	Technical Background	2
2.1	HTTP Routing and the Host Header	2
2.2	DNS Resolution and Subdomain Structure	2
2.3	Content Discovery via Directory/File Brute-Forcing	2
3	Mode Anatomy	3
3.1	dir — Directory/File Enumeration	3
3.2	dns — Subdomain Enumeration	3
3.3	vhost — Virtual Host Enumeration	3
3.4	fuzz — Generic FUZZ Substitution	4
4	Practical Usage & Flags	4
4.1	dir Mode — Basic to Advanced	4
4.2	dns Mode — Basic to Advanced	4
4.3	vhost Mode — Basic to Advanced	5
4.4	fuzz Mode — Basic to Advanced	5
5	Tools & Methodology	6
5.1	Checklist	7
6	Bypasses & Edge Cases	7
	Appendices	8

1 Introduction

- Gobuster is a high-performance, Go-based brute-forcing tool written by Christian Mehlmauer and OJ Reeves, used to discover content and hosts that are not linked anywhere and therefore invisible to normal browsing or crawling.
- It operates in distinct **modes**, each targeting a different discovery surface: `dir` (files/directories), `dns` (subdomains), `vhost` (virtual hosts sharing one IP), `fuzz` (generic keyword substitution), plus `s3`, `gcs` (cloud storage buckets) and `tftp`.

2 Technical Background

2.1 HTTP Routing and the Host Header

- A single web server (one IP, one port) can serve multiple distinct sites. The server decides which site's content to return based on the `Host` header sent in the HTTP request, not the IP address dialed.
- This is called **name-based virtual hosting**. Apache implements it via `<VirtualHost>` blocks matched on `ServerName/ServerAlias`; Nginx via `server_name` directives inside `server {}` blocks.
- If a hostname is never advertised through public DNS or links, the only way to discover it is to guess candidate hostnames and observe whether the server responds differently — this is exactly what `vhost` mode automates.

2.2 DNS Resolution and Subdomain Structure

- Subdomains (e.g. `staging.example.com`) are DNS records (typically `A`, `AAAA`, or `CNAME`) under a parent zone. They can exist and resolve publicly even if never linked from the main site.
- `dns` mode brute-forces subdomain names and performs a live DNS resolution for each candidate; a response means the record exists.
- **Wildcard DNS** is the main technical pitfall: some zones resolve *any* subdomain (even nonsense ones) to a catch-all IP, which would otherwise flood results with false positives. Gobuster detects wildcards automatically and Gobuster's `--wildcard` flag forces the scan to continue anyway once detected.

2.3 Content Discovery via Directory/File Brute-Forcing

- `dir` mode requests each wordlist entry as a path against the target and inspects the HTTP status code (and optionally response length) to decide if the resource exists.
- Frameworks frequently leave predictable artifacts reachable but unlinked: `/.git/HEAD`, `/backup.zip`, `/admin`, `/api/v1/`, `/.env` — none of these appear in any menu, yet all are directly requestable.

3 Mode Anatomy

3.1 dir — Directory/File Enumeration

- Iterates a wordlist over URL paths, classifies responses by status code (default positive set: 200, 204, 301, 302, 307, 401, 403).
- Attack surface: any path-addressable resource — backup files, config files, forgotten admin routes, API versioning paths (`/api/v2/`), source control artifacts.

How It Works (Mechanism & Why It Succeeds)

Servers do not enumerate their own file tree to visitors; a nonexistent path simply returns 404, and an existing-but-unlinked path returns 200/301 identically to a linked one. There is no cryptographic or access-control difference between “linked” and “unlinked” content — only obscurity. Any developer who ships `/backup.sql` into the webroot without an authentication check is trusting that nobody guesses the name; brute-forcing defeats that trust trivially.

3.2 dns — Subdomain Enumeration

- Brute-forces subdomain labels and resolves each against the authoritative or a specified resolver.
- Attack surface: any subdomain that exists as a DNS record, whether or not a certificate or working service sits behind it.

How It Works (Mechanism & Why It Succeeds)

DNS zones are queried, not crawled — a client can ask “does `X.example.com` exist?” directly, and the resolver answers truthfully regardless of whether `X` is publicly advertised anywhere. Organizations that provision `staging.example.com` or `old-admin.example.com` in DNS for internal convenience, assuming “nobody will find it because it isn’t linked,” are relying on the same false obscurity as unlinked `dir` content.

3.3 vhost — Virtual Host Enumeration

- Sends requests to a single IP/URL while iterating the `Host` header value, then compares response length/status/body to a baseline to detect a genuinely different site.
- Attack surface: any vhost configured server-side but never given a public DNS record — e.g. an internal admin console reachable only if you already know its hostname.

How It Works (Mechanism & Why It Succeeds)

Name-based virtual hosting means the web server, not DNS, holds the authoritative list of what hostnames it will serve. A request can reach the correct *virtual host block* purely by carrying the right `Host` header, even if that hostname resolves nowhere in public DNS or resolves only to a different IP entirely. Admins who assume “this vhost is safe because it isn’t in DNS” are wrong: DNS and vhost configuration are two independent, decoupled mechanisms.

3.4 fuzz — Generic FUZZ Substitution

- Replaces a literal FUZZ marker anywhere in the URL, headers, or POST body with wordlist entries. Not restricted to a fixed position like `dir/vhost`.
- Attack surface: GET/POST parameter values, custom headers, arbitrary body fields — anywhere user-controlled input might reach a sensitive sink (SSRF-prone `format=` or `url=` parameters, IDOR-prone numeric IDs, auth-bypass headers).

How It Works (Mechanism & Why It Succeeds)

Because the position of the payload is entirely user-defined via the FUZZ token, this mode has no assumption about *where* untrusted input matters — it mirrors the general principle that any attacker-influenced value (not just the URL path) can be a discovery/attack vector if the backend logic branches on it (e.g. a `format` parameter selecting a code path that fetches a URL server-side).

4 Practical Usage & Flags

4.1 dir Mode — Basic to Advanced

Basic sweep with a common wordlist:

```
gobuster dir -u https://target.com -w /usr/share/wordlists/dirb/common.txt
```

Add extensions and filter noise by status code:

```
gobuster dir -u https://target.com \
-w /usr/share/wordlists/SecLists/Discovery/Web-Content/raft-large-words.txt \
-x php,bak,old,zip,tar.gz,env \
-s 200,204,301,302,307,401,403 -t 50
```

Authenticated brute-force behind a session cookie, following redirects, ignoring TLS errors:

```
gobuster dir -u https://target.com/account \
-w raft-large-words.txt \
-c "session=<SESSION>" -r -k -o results.txt
```

Advanced: hide soft-404 pages that all return 200 with identical body length:

```
gobuster dir -u https://target.com -w common.txt --exclude-length 9265
```

4.2 dns Mode — Basic to Advanced

Basic subdomain sweep:

```
gobuster dns -d target.com \
-w SecLists/Discovery/DNS/subdomains-top1million-5000.txt -t 50
```

Advanced: reveal CNAMEs (useful for spotting dangling-CNAME subdomain-takeover candidates), force through a wildcard zone, use a specific resolver to dodge local rate limiting:

```
gobuster dns -d target.com -w subdomains-top1million-20000.txt \
-r 1.1.1.1 -c --wildcard -t 100
```

4.3 vhost Mode — Basic to Advanced

Basic vhost sweep against a known IP, appending the base domain to each wordlist entry:

```
gobuster vhost -u http://10.10.10.5 -w subdomains.txt --append-domain
```

Advanced: skip TLS verification, output results, then manually confirm a genuine hit by diffing response length against the baseline (vhost mode lacks `-s/-b` status-code filters, so false positives from catch-all vhosts are common and must be checked by hand):

```
gobuster vhost -u https://10.10.10.5 -w subdomains-top1million-20000.txt \
--append-domain -k -o vhost_results.txt
```

Realistic raw request that gobuster is effectively automating, and the request you would replay manually in Burp Repeater once a candidate hit is found:

```
GET / HTTP/1.1
Host: intranet.target.com
Cookie: session=<SESSION>
User-Agent: Mozilla/5.0
Connection: close

HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 4821
Set-Cookie: PHPSESSID=<REDACTED>; path=/
```

4.4 fuzz Mode — Basic to Advanced

Parameter value fuzzing:

```
gobuster fuzz -u "https://target.com/export?format=FUZZ" \
-w format_payloads.txt -b 404
```

POST body fuzzing (e.g. probing an IDOR-prone identifier):

```
gobuster fuzz -u https://target.com/api/invoice \
-m POST -d "invoice_id=FUZZ" \
-H "Content-Type: application/x-www-form-urlencoded" \
-H "Cookie: session=<SESSION>" -b 403
```

Custom header fuzzing (testing for header-based access-control bypass):

```
gobuster fuzz -u https://target.com/admin \
-H "X-Forwarded-For: FUZZ" -w bypass_ips.txt
```

Realistic HTTP request rendered from the above (auth-bypass attempt via trusted-header spoofing):

```
GET /admin HTTP/1.1
Host: target.com
X-Forwarded-For: 127.0.0.1
Cookie: session=<SESSION>
Connection: close

HTTP/1.1 200 OK
Content-Length: 6120
```

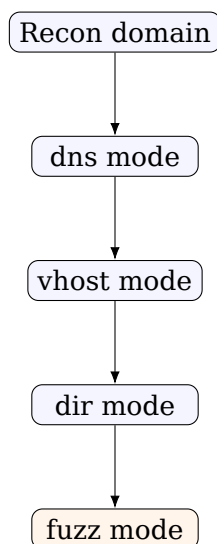


Figure 2: Typical discovery escalation order — widen the attack surface (*dns*, *vhost*) before drilling into content (*dir*) and finally parameter-level logic (*fuzz*).

5 Tools & Methodology

- **SecLists** — the standard wordlist repository (Discovery/Web-Content for *dir*, Discovery/DNS for *dns*/*vhost*).
- **Burp Suite** — send confirmed hits to Repeater for manual follow-up; Burp’s built-in Content Discovery / Intruder can cross-validate gobuster findings.
- **subfinder** / **amass** — passive subdomain enumeration (certificate transparency logs, search engines) to seed or supplement gobuster’s active *dns* brute-force.
- **httpx** — mass HTTP probing of resolved subdomains/vhosts to quickly triage which are alive and worth deeper inspection.
- **ffuf** — a common alternative/companion to *fuzz* mode with more flexible multi-wordlist and recursive fuzzing.

Methodology (map → identify → confirm → escalate), adapted to discovery work:

1. **Step 1 — Passive recon.** Gather known subdomains via *subfinder*/certificate transparency before touching the target directly.
2. **Step 2 — Active DNS sweep.** Run *dns* mode to catch anything passive sources missed; note wildcard behavior early.
3. **Step 3 — VHost sweep.** Once IPs are known, run *vhost* mode against each IP to surface hosts with no public DNS record at all.
4. **Step 4 — Content discovery.** Run *dir* mode against every live host found, using a stack-appropriate wordlist once the technology is fingerprinted.
5. **Step 5 — Parameter/logic fuzzing.** Use *fuzz* mode on interesting endpoints found in Step 4 to probe parameter values, headers, and body fields.

Mindset: treat every “clean” site as incomplete until content discovery has run. Common tells that discovery will pay off: a site with a CMS (predictable admin paths), recently migrated infrastructure (stale staging vhosts), or an API with visible versioning (*/v1/* implies */v2/* may also exist). Common misconception: a 404 for the root path does not mean the host has no hidden content — it only means that one path is unmapped.

5.1 Checklist

- ☐ Confirm scope/authorization before any brute-force traffic is sent.
- ☐ Run passive subdomain recon before active dns brute-force.
- ☐ Check for wildcard DNS before trusting dns mode results.
- ☐ Run vhost mode against every known IP, not just the primary hostname.
- ☐ Cross-check vhost hits by comparing response length/body, not status code alone.
- ☐ Tune dir wordlist and extensions to the fingerprinted stack (CMS, framework).
- ☐ Set sane thread counts (-t) to avoid unintentional denial-of-service or IDS/WAF bans.
- ☐ Fuzz interesting parameters/headers found during content discovery with fuzz mode.
- ☐ Log all output (-o) for reporting and later diffing between engagement phases.

6 Bypasses & Edge Cases

- **Wildcard DNS false positives.** A zone that resolves any subdomain to a catch-all IP will make every dns-mode guess look like a hit unless --wildcard handling and manual verification are used.
- **Vhost catch-all false positives.** Servers without a proper default vhost return the same page for every Host header tried, inflating apparent hits — always diff response length/body against a known-bad control hostname.
- **Case sensitivity.** Some servers/filesystems are case-sensitive (Linux) while others are not (Windows/IIS); running a wordlist with mixed casing variants can surface additional hits on case-sensitive targets.
- **Rate-limit evasion vs. noise trade-off.** Lower thread counts (-t) and added --delay reduce detection risk but slow discovery; aggressive threading risks WAF bans or denial-of-service on fragile targets.
- **Forgetting extensions.** dir mode does not append extensions by default — /config will be found but /config.php will not unless -x php is set explicitly.
- **No recursion.** Gobuster does not recurse into discovered directories automatically; a fresh dir run must be issued against each newly found directory.

Appendices

A. Cheatsheet

Mode	Flag	Effect
dir	-u	Target URL
dir	-w	Wordlist
dir	-x	Extensions to append (php,bak,zip)
dir	-s / -b	Status codes to show / blacklist
dir	-r	Follow redirects
dir	-k	Skip TLS certificate verification
dir	-c	Cookies for the requests
dir	-H	Custom header, e.g. -H "Authorization: Bearer <TOKEN>"
dir	--exclude-length	Hide responses of a given byte length (soft-404 filtering)
dns	-d	Target domain
dns	-r	Custom DNS resolver
dns	-c	Show CNAME records
dns	-i	Show resolved IPs
dns	--wildcard	Force continued operation when wildcard DNS is found
vhost	-u	Target URL/IP
vhost	-w	Vhost wordlist
vhost	--append-domain	Append base domain to each wordlist entry
vhost	-k	Skip TLS certificate verification
fuzz	-u	URL containing the FUZZ marker
fuzz	-m	HTTP method (e.g. POST)
fuzz	-d	POST body containing FUZZ
fuzz	-H	Header containing FUZZ
global	-t	Thread count
global	-o	Output file
global	--delay	Delay between requests per thread